PRODUKTKONZEPT / VERSION 1.0

TermDock

Persistent Workspace & Terminal Manager
Windows Desktop · WSL-first · CLI-native

Produktvision
Eine minimalistische Windows-Desktop-Anwendung zur Verwaltung langlebiger Terminal-Sessions, projektbezogener Workspaces und CLI-Entwicklungswerkzeuge. Claude Code, Codex und andere Harnesses werden als normale CLI-Programme ausgeführt; TermDock entwickelt keine eigene Agent-Laufzeit.

PROJEKT	ENTSCHEIDUNG
Repository	/home/nic/custom_repos/termdock
Oberfläche	Minimalistisch wie VS Code / Cursor
Standardumgebung	WSL, Windows-Shell optional
Priorität	Zuverlässiger Terminal- und Session-Manager
Status	Abgestimmtes V1-Konzept; Implementierung noch ausstehend

Stand: 13. November 2026 | Dokumenttyp: Umsetzungsspezifikation / Planung

1. Zielbild und Produktgrenze

TermDock organisiert Projektkontexte und Terminal-Sessions. Der Mehrwert liegt in der schnellen Navigation, langlebigen Prozessen, einem rekonstruierbaren Layout und der konsistenten Verwaltung mehrerer Fenster. Es ersetzt weder IDE noch Agent-CLI.

Nicht-Ziele von V1

- Keine eigene Claude-Code-, Codex- oder sonstige Agent-Engine; keine Prompt-Orchestrierung.
- Kein zuverlässiges Agent-Status-Dashboard oder zentraler Agent-Genehmigungsworkflow ohne spätere tool-spezifische Adapter.
- Kein SSH-/Remote-Host-Management und keine geräteübergreifende Synchronisierung.
- Keine vollautomatische Git-Worktree-Orchestrierung.

2. Verbindliche V1-Anforderungen

Bereich	Abgestimmtes Verhalten
Plattform / Design	Eigenständige Windows-App; reduzierte Oberfläche im Stil VS Code/Cursor.
Shell-Backends	WSL als Standard; Windows-Shells optional und mit separat definierter Prozesspersistenz.
Workspaces	Gruppen mit mehreren Projektordnern/Repositories; auch Sessions ohne Gruppe.
Startverhalten	Nur explizit als Autostart markierte Session-Profile werden automatisch ausgeführt.
Navigation / Layout	Tabs; 2–4 Split-Panes; mehrere Fenster; vollständiges Layout wiederherstellen.
Gemeinsame Session	Dieselbe Session in mehreren Fenstern sichtbar, aber immer nur eine aktive Schreibansicht.
Archiv	Beim Archivieren Prozess beenden; Metadaten und begrenzten Verlauf behalten.
Verlauf	Dauerhaftes Logging standardmäßig auf 10 MiB je Session begrenzt, abschaltbar.
Startprofile	Konfigurierbare Startbefehle und Arbeitsverzeichnisse; Agents laufen nativ als CLI.
App schließen	Dialog zur Auswahl: Oberfläche schließen und Sessions behalten oder Sessions beenden.

3. Bedienmodell

Sidebar: Workspaces mit darin liegenden Projekten/Sessions, zusätzlicher Bereich für gruppenlose Sessions sowie Archiv. Arbeitsfläche: Tabs und Split-Panes. Statusleiste: Backend/Distribution, aktiver Kontext und Sessionzustände. Einstellungen: WSL-Distribution, Standardshell, Shortcuts, Startprofile, Verlauf und Datenschutz.

Mehrere Projektverzeichnisse pro Workspace sind zulässig. Jede neue Session erhält ein explizites Arbeitsverzeichnis; falls keines gewählt wird, gilt das konfigurierte Standardverzeichnis der Gruppe beziehungsweise Shell.

4. Technischer Architekturvorschlag

Ein zentraler Core verwaltet Sessionidentitäten und Prozesse. Ein oder mehrere Desktop-Fenster sind lediglich Clients dieses Core. Damit werden doppelte Prozesse und widersprüchliche Zustände vermieden.

Schicht	Vorgeschlagene Technologie / Zuständigkeit
Desktop	Electron, React, TypeScript; Fenster, Sidebar, Tabs, Split-Layouts und Einstellungen.
Terminalansicht	xterm.js; Terminalemulation, Resize, Tastatur, Zwischenablage und Scrollback.
Core	Session Registry, Lifecycle, Eingabe-Lock, Backend-Steuerung, lokale IPC.
WSL Backend	WSL-Ausführung mit tmux als persistenter Session-Multiplexer.
Windows Backend	ConPTY/PTy-Anbindung (z. B. node-pty), abhängig vom Lebenszyklus des Core.
Storage	SQLite für Konfiguration/Metadaten; begrenzter Output-Verlauf als separate persistente Ablage.

Architekturprinzipien

- Eine Session, beliebig viele Views: Sessionobjekt und sichtbarer Tab beziehungsweise Split-Pane sind getrennt.
- Eine schreibende Ansicht: Der Core verwaltet die Eingabeberechtigung atomar und entscheidet über Terminal-Resize.
- Backend-Abstraktion: WSL und Windows haben dieselbe UI, aber eigenständige Prozess- und Wiederanbindungslogik.
- Keine Agent-Sonderbehandlung in V1: Claude Code, Codex und ähnliche Programme werden über generische CLI-Startprofile ausgeführt.

5. Datenmodell

Entität	Bedeutung
Workspace	Logische Gruppe; enthält Projekte, Sessions und optionale Startprofile.
Project	Referenziert ein Arbeitsverzeichnis / Repository, Backend und optionale Git-Informationen.
Session	Eindeutige logische Terminalinstanz, Prozess-/tmux-Referenz, Arbeitsverzeichnis, Status.
View	Tab oder Split-Pane als Darstellung einer Session; mehrere Views pro Session möglich.
LaunchProfile	Shell/Backend, Verzeichnis, Befehl, Startparameter, Autostart und Umgebungsregeln.
HistoryRecord	Begrenzte Ausgabedaten mit Zeitstempeln/Sessionbezug und Suchmetadaten.

Hinweis: Geheime Zugangsdaten gehören nicht im Klartext in Startprofile oder Logs. Geeignete Mechanismen sind bestehende CLI-Logins, geschützte Umgebungsvariablen oder OS-basierte Secret-Stores.

6. Session-Lifecycle und Neustartverhalten

Zustand	Semantik
Starting	Session/Backend wird angelegt oder wiederverbunden.
Running	Verwalteter Shell-/Terminalprozess lebt und ist ansprechbar.
Detached	Prozess läuft, aber aktuell ohne angehängte View.
Exited	Shell beziehungsweise verwalteter Prozess ist kontrolliert beendet.
Interrupted	Prozess ging unerwartet verloren oder konnte nicht wiedergefunden werden.
Archived	Kein aktiver Prozess; archivierte Metadaten und verfügbarer Verlauf bleiben erhalten.

Verhalten bei unterschiedlichen Ereignissen

Ereignis	Erwartung
Tab/Workspace wechseln	Prozess läuft unverändert weiter.
UI schließen, Sessions behalten	Core/Backend bleiben so weit technisch möglich aktiv; Reattach beim nächsten Start.
UI schließen, Sessions beenden	Verwaltete Sessions kontrolliert stoppen.
Windows/WSL wird neu gestartet	Layout/Metadaten rekonstruieren; verlorene Prozesse als Interrupted statt Running anzeigen.
Autostart	Ausschließlich explizit markierte Profile. Doppelte Starts durch Reattach vermeiden.

Wichtige Grenze: tmux schützt nicht vor dem Beenden der WSL-Instanz. Die Wiederherstellung einer Oberfläche ist deshalb nicht gleichzusetzen mit dem Fortsetzen eines gestoppten Prozesses.

7. Mehrfenster-Betrieb und Eingabefokus

- Mehrere Views können dieselbe Session live anzeigen.
- Genau eine View besitzt das Schreibrecht; ein Klick kann dieses über den Core übernehmen.
- Der Core serialisiert Eingaben und verwendet die schreibende View als Quelle der maßgeblichen Terminalgröße.
- Ein Fensterwechsel oder Schließen einer View beendet nicht automatisch die Session.

8. Verlauf, Suche, Archiv und Datenschutz

Persistenter Verlauf: standardmäßig ein auf 10 MiB je Session begrenzter Ringbuffer. Das Limit gilt für dauerhaft gespeicherte Outputdaten und ist unabhängig vom flüchtigen Display-Scrollback. Das Archiv bewahrt den begrenzten Verlauf, sofern Logging aktiv war. Ein Archiv ermöglicht keine magische Wiederaufnahme eines beendeten Programms.

Suche läuft nur über gespeicherte Daten. Escape-Sequenzen und sensible Informationen erfordern besondere Behandlung. Logging ist pro Session abschaltbar. Beim Löschen werden auch Verlaufsdaten entfernt.

9. CLI-Startprofile und Agent-Nutzung

Ein Startprofil definiert, mit welchem Backend, in welchem Projektverzeichnis und mit welchem Kommando eine Session gestartet wird. Konkrete Beispiele: `claude`, `codex`, `dotnet watch`, eine Shell für Git. TermDock behandelt diese Programme in V1 gleichwertig und garantiert ausschließlich Terminal-Interaktivität, Prozessstatus und Sessionverwaltung.

Später optional

- Tool-spezifische Adapter für Agent-Zustände, Benachrichtigungen oder Genehmigungsdialoge.
- Git-Worktree-Erstellung für parallel schreibende Agents; nicht für jede Shell erzwingen.
- Lokale CLI/API zur Fernsteuerung von Workspaces und Sessions.

10. V1-Abnahmekriterien

01. Workspace mit mehreren Projektordnern anlegen und öffnen.
02. Neue WSL-Shell startet im erwarteten Arbeitsverzeichnis.
03. Claude Code/Codex funktioniert als interaktive CLI in einem Startprofil.
04. Tabwechsel lässt die Session und den Prozess aktiv.
05. Dieselbe Session kann in zwei Fenstern angezeigt werden; nur eine Ansicht sendet Eingaben.
06. 2–4 Split-Panes verhalten sich korrekt bei Resize.
07. Nur markierte Autostart-Profile werden automatisch ausgeführt.
08. Workspace-, Tab- und Split-Layout wird beim UI-Neustart wiederhergestellt.
09. Abgebrochene WSL-Sessions werden als Interrupted markiert.
10. Archivieren beendet den Prozess und erhält den zulässigen Verlauf.
11. Das Speicherlimit für History greift auch bei hoher Terminalausgabe.
12. Gruppenlose Sessions lassen sich anlegen, verschieben, archivieren und löschen.
13. Der Schließen-Dialog bietet verständliche Optionen zum Session-Lifecycle.
14. Mehrere UI-Fenster verursachen weder doppelte Sessions noch doppelte Autostarts.
15. Windows-Shells funktionieren mit erkennbaren Persistenzgrenzen.

11. Technische Risiken und Validierung

Risiko	Validierung / Gegenmaßnahme
tmux-Reattach / Resize	Proof of Concept: neue Session, detach, reattach, mehrere Views und Eingabefokus.
Windows-Core-Lifecycle	Core unabhängig vom Fenster betreiben; Persistenzversprechen präzise dokumentieren.
Rich TUI-Kompatibilität	Interaktive CLIs mit Farben, Escape-Sequenzen, Clipboard und Terminalgröße testen.
History/Sicherheit	Speicherlimits, Logging-Ausnahmen, Log-Löschung und Suche gezielt testen.
Abstürze / Reconcile	Bei Startup gespeicherte Session-Registry mit tatsächlich laufenden Backends abgleichen.

12. Implementierungsreihenfolge

Phase 0 – Technischer Proof of Concept

WSL-tmux-Session starten, in xterm.js anzeigen, detach/reattach durchführen und zwei gleichzeitig geöffnete Fenster mit synchronem Output und exklusivem Eingaberecht testen. Keine Produkt-UI erforderlich.

Phase 1 – Kernsystem

Core mit stabilen Session-IDs, WSL-Backend, Lifecycle-Regeln, Zustandserkennung und lokaler IPC. Anschließend Persistenz der Workspace-/Session-Metadaten.

Phase 2 – Desktop-Erlebnis

Minimalistische Sidebar, Tabs, 2–4 Split-Panes, Shortcut-Navigation und vollständige Layout-Wiederherstellung.

Phase 3 – Produktfunktionen

Startprofile, Autostart-Regeln, Gruppen ohne Projektbindung, Archiv/Löschung, History-Ringbuffer, Suche, Einstellungsdialoge und Windows-Backend.

Phase 4 – Stabilisierung

Abnahmetests, Ausfall- und Neustartszenarien, Performance bei vielen Sessions, Logging-/Security-Tests und Windows-Paketierung.

13. Bewusst nach V1 verschoben

Ausbaustufe	Funktionen
V1.1	Automatische Worktree-Verwaltung, erweiterte Templates und lokale CLI/API.
V2	Optionale Agent-Statusadapter, strukturierte Freigabe- und Benachrichtigungsfunktionen, SSH/Remote-Hosts.
Später / offen	Geräteübergreifende Synchronisation und weitere externe Integrationen.

14. Offene technische Detailentscheidungen

Der V1-Produktumfang ist beschlossen. Vor Implementierung sollten noch die konkreten IPC-Contracts, die Prozesslebensdauer des Windows-Backends, das genaue Historienformat, das Verhalten bei Core-Neustart und die Semantik einer automatischen Session-Rekonstruktion in kurzen Architecture Decision Records festgehalten werden.

Fazit: V1 ist als fokussierter Terminal- und Workspace-Manager spezifiziert. Die größte technische Unsicherheit ist nicht die Benutzeroberfläche, sondern das zuverlässige Reattach- und Mehrfenster-Verhalten – genau dieses sollte zuerst prototypisch nachgewiesen werden.

Planungsstand: Konzeptdokument, kein Implementierungsnachweis. Das Projektverzeichnis wurde durch die Erstellung dieses PDFs nicht angelegt.